

Citation and Reference Tree Generator

Independent Study, Spring 2026

Bhavani Chalasani
bhavani.chalasani@students.iiit.ac.in
IIIT Hyderabad

Nishita Kannan
nishita.kannan@students.iiit.ac.in
IIIT Hyderabad

Kamalakar Karlapalem
kamal@iiit.ac.in
IIIT Hyderabad

Abstract—This project presents the design and implementation of an end-to-end citation and reference tree generator that accepts a research paper PDF, discovers semantically related prior and subsequent works across multiple research-focused APIs, and renders HTML visualizations of the corresponding citation and reference tree for user exploration. It integrates PDF text extraction via Apache Tika, multi-source metadata retrieval from Semantic Scholar, OpenAlex, and arXiv, and uses both heuristic and embedding-based methods for relevance scoring and filtering. Additionally, large language models (LLMs) are used to generate explanations of parent-child relationships within the trees. The implementation emphasizes robustness under real-world constraints such as API rate limits, noisy or incomplete metadata, and variability in research paper formats. We outline a deployment pipeline, and we also discuss limitations and future scope for improving coverage, accuracy, and performance.

Index Terms—citations, references, semantic similarity, LLM summarization, PDF information extraction, HTML rendering

I. Introduction

Understanding research often means knowing where an idea came from (its references) and how it influenced later work (its citations). While existing academic search tools can find papers, they usually don't provide a clear and easy way of understanding how an idea came to be and how it influenced later work. Therefore, an easy-to-navigate tree that brings together these connections with meaningful content context is essential.

This project addresses that gap through a practical system that:

- 1) accepts a paper as input,
- 2) builds both a reference tree and a citation tree,
- 3) enriches links with semantic and LLM-generated improvement explanations, and
- 4) renders interactive outputs through a web interface.

II. Problem Statement and Objectives

II-A. Problem

Given one paper, construct two bounded trees:

- **Reference tree:** previous work cited by the paper and recursively by selected descendants.
- **Citation tree:** later work citing the paper and recursively selected descendants.

The challenge is that scholarly APIs vary in completeness and identifiers, PDF text is noisy, and strict filtering may collapse trees for newer or sparse-coverage papers.

II-B. Objectives

The project objectives were:

- PDF uploading of a research paper,
- multi-source retrieval with fallback resolution,
- relevance and semantic based child selection,
- explanation generation for parent-child paper relationships,
- interactive HTML outputs.

III. System Overview

The pipeline is organized into five stages.

III-A. Stage 1: Root Paper Extraction

The root PDF is parsed to recover:

- title,
- abstract,
- references section text,
- arXiv identifier (if present),
- full extracted text.

III-B. Stage 2: Cross-Source Root Identification

The extracted information about the input paper is checked across multiple research APIs to find the correct match. This process includes:

- Looking up the paper directly using its arXiv ID when available,
- Searching by title if no ID match is found,

- Comparing titles to find the closest match,
- Selecting only high-confidence matches to avoid incorrect results.

This step ensures that the system starts with the correct version of the paper (the input paper may be outdated), which helps improve the accuracy of the citation and reference tree built later.

III-C. Stage 3: Tree Expansion

After identifying the root paper, the system expands the tree in two directions: backward (references) and forward (citations). This helps capture both the origins of the idea and its later impact.

- **Reference expansion:** The system first collects the list of papers referenced by the root paper using the available APIs. If this information is missing or incomplete, it falls back to searching for similar papers based on the title and selects the closest matches as substitutes.
- **Citation expansion:** The system finds papers that cite the root paper. If no citing papers are returned (which can happen due to API limitations or missing data), it uses a fallback approach by retrieving related or similar papers to approximate citation relationships.

Both strategies include filtering and matching steps to ensure that only relevant and high-confidence papers are added to the tree. This approach allows the system to build a more complete and useful dependency network even when data from APIs is limited or noisy.

III-D. Stage 4: Postprocessing and Enrichment

For the selected tree nodes:

- attempt PDF download,
- extract full text and improved abstracts,
- compute parent-child semantic similarity,
- generate an improvement explanation for each edge.

III-E. Stage 5: Output

The system writes:

- JSON tree artifacts for machine use,
- self-contained HTML views for human exploration.

IV. Implementation Details

IV-A. Core Components

TABLE I
SYSTEM CORE COMPONENTS

Component	Responsibility
Ingestion	Accept PDF of the input paper.
PDF extractor	Use Apache Tika for text/metadata extraction.
API clients	Query Semantic Scholar, OpenAlex, and arXiv.
Tree builder	Expand trees with depth/size constraints and fall-back logic.
ML layer	Compute embeddings; summarize and generate explanations via LLM.
Renderer	Produce interactive HTML with searchable details.

IV-B. Configuration and Runtime Controls

The implementation is environment-driven with controls for:

- depth and breadth (MAX_DEPTH, MAX_PAPERS, MAX_CHILDREN),
- retrieval limits (API_REFERENCE_LIMIT, API_CITATION_LIMIT),
- relevance threshold (MIN_RELEVANCE),
- LLM behavior (LLM_EXPLANATIONS, LLM_PROVIDER, budget),
- cleanup policy (DELETE_PDFS_AFTER_USE).

V. Methodology

AI tools were used to assist with understanding of different concepts and debugging. All outputs were reviewed, edited, and verified by the authors.

V-A. Metadata Extraction and Root Normalization

Root extraction uses Tika metadata fields and text heuristics. A lock is used around Tika parser calls to avoid thread races during startup or concurrent parsing, since both the reference and citation trees get created in parallel (based on the runtime control set when configuring the generator). If API lookup fails, a local paper object is created from extracted metadata so processing can continue.

V-B. Reference Tree Expansion

Reference expansion prioritizes true references:

- 1) fetch references by source-specific IDs over the different APIs,
- 2) fallback to title-based source resolution when IDs are unavailable,
- 3) fetch references from the different APIs based on other identifiers (such as using the arXiv id to find the paper via the Semantic Scholar API).

Candidates are filtered to keep only useful papers. This includes:

- removing papers with low relevance scores,
- removing duplicates or papers that match the current or root paper,
- keeping only papers that are likely open access,
- ensuring the paper is similar enough in meaning.

V-C. Citation Tree Expansion

Citation expansion uses fetching from Semantic Scholar and OpenAlex after resolving source IDs. If direct citations are absent, the system switches to a related-paper fallback pool and labels such edges as `related` instead of forcing `citation`. This preserves semantic transparency.

Additional filtering also considers the publication year:

- papers published before the parent paper (for example, if the paper has multiple versions) are removed, since a citation should come after,
- this helps keep the citation tree logically correct and avoids incorrect links.

V-D. Relevance and Similarity Scoring

Candidate relevance combines word overlap, title overlap, category match, citation count, and relation type:

$$\begin{aligned} \text{Rel}(s, c) = & 0.35 \cdot \text{WordOverlap} \\ & + 0.15 \cdot \text{TitleOverlap} \\ & + 0.20 \cdot \text{CategoryOverlap} \\ & + \min\left(\frac{\text{Citations}}{500}, 0.15\right) \\ & + \text{RelationBonus} \end{aligned} \quad (1)$$

where `RelationBonus` gives higher weight to direct references/citations and lower weight to loosely related papers.

Semantic similarity is computed using cosine similarity over embeddings:

$$\text{Sim}(a, b) = \frac{\mathbf{e}_a \cdot \mathbf{e}_b}{\|\mathbf{e}_a\| \|\mathbf{e}_b\| + 10^{-9}} \quad (2)$$

Ranking strategy:

- references: first filter papers using a relevance score, then compute semantic similarity for the remaining candidates, and finally select the most similar papers,
- citations: first filter using relevance and validity checks, then rank by publication year (older first), followed by semantic similarity and relevance.

V-E. Postprocessing and Enrichment

After structure is built, the pipeline hydrates content only for selected nodes:

- 1) download missing PDFs if possible,
- 2) extract full text and metadata,
- 3) backfill missing abstracts from metadata if needed,
- 4) compute parent-child similarity,
- 5) generate parent-child improvement explanations.

VI. LLM Layer Design for Improvement Generation

VI-A. Provider Strategy

The system supports Groq and Gemini under an OpenAI compatible client interface. In `auto` mode, provider fallback is used for quota/rate/auth failures.

VI-B. Call Budgeting and Thread Safety

LLM usage is limited by a set budget for each run. The system keeps track of which provider is used and how much of the budget is spent separately for each thread, so reference and citation processes running at the same time do not interfere with each other.

VI-C. Summary and Explanation Generation

For each paper pair:

- chunk the text used for summary generation,
- build compact summaries from chunked text extraction via chunk based input and compiling from the LLM,
- prompt for concrete parent-child technical relationship using the summaries of the parent and child,
- output concise improvement explanation,
- cache summaries and pair explanations.

If LLM output is unavailable, deterministic fallback explanations are generated to avoid blank fields.

VII. Caching, Rate Limiting, and Reliability

VII-A. Disk Cache

A lightweight JSON-based disk cache is used to store API search and lookup results, paper summaries, extracted abstracts, and generated explanation texts. Each entry is saved with a timestamp and expires after a fixed time-to-live (TTL). Cache files are indexed using hashed keys to ensure efficient storage and retrieval. This helps reduce repeated API calls and improves performance while handling occasional read/write failures safely.

VII-B. Global Request Gate

All HTTP requests are controlled through a global request gate that limits both concurrency and request rate. A semaphore restricts the number of simultaneous requests, while a per-source timing mechanism ensures that consecutive calls to the same API are spaced out. This prevents exceeding

API rate limits and reduces the risk of request failures due to bursts.

VII-C. Open-Access Filtering

To improve downstream PDF hydration quality, candidates with DOI-only closed signals (papers that are not free or open to download) are filtered unless open-access evidence exists (arXiv ID, OA flags, or direct PDF URL).

VIII. Interactive Visualization

VIII-A. Renderer

Each tree is rendered to a standalone HTML artifact with embedded JSON and JavaScript. Features include:

- tree traversal with depth indentation,
- node detail panel (title, authors, abstract, categories),
- improvement explanation and similarity bar,
- quick paper link-out (URL/DOI/PDF),
- summary statistics (paper count, edge count, year range),
- tab switching between reference and citation views.

IX. Deployment and Operations

IX-A. Containerization

This app runs inside a lightweight Python 3.12 Docker container. It also installs a small Java runtime because Apache Tika needs Java to process PDFs. In production, Gunicorn uses 1 worker by default to keep memory usage low.

IX-B. Hosted Deployment

The app is deployed on Hugging Face Spaces, on the free plan. Low-memory mode is turned on by default. API keys are stored as secure environment secrets in Render, not hardcoded in the app.

IX-C. Notes

- During the first run, there is an initial delay because the Tika engine starts up and the embedding model is loaded.
- Long jobs can take tens of minutes for deep expansion.

X. Limitations

- API coverage is incomplete and can vary by source, model, and publication recency.
- Related-paper fallback may introduce semantically plausible but non-citation edges.
- LLM explanations depend on extracted text quality and prompt precision.
- Runtime is bounded by network latency, rate limits, and PDF parse complexity.
- No formal human annotation study was done for explanation quality.

XI. Ethical and Practical Considerations

- API keys are externalized as environment secrets, not hardcoded.
- Open-access preference reduces dependency on inaccessible full text.
- Explanations are assistive summaries and should not replace source-paper reading.
- The generated relationship text should be treated as a probabilistic interpretation, not a definitive causality.

XII. Conclusion

This project delivered a complete, deployable research-paper exploration system that combines scholarly API integration, PDF extraction, graph expansion, semantic ranking, LLM interpretation, and interactive visualization. The implementation balances quality and robustness with practical deployment constraints and provides a strong foundation for future large-scale scholarly graph intelligence workflows.

XIII. Future Work

- 1) Add proper evaluation (not just manual human evaluation) by checking how accurate the parent-child links are and how reliable the explanations are.
- 2) Include more research sources such as Crossref, DBLP, and PubMed.
- 3) Show confidence levels and uncertainty in the user interface.
- 4) Allow users to download trees or view previously generated trees later.
- 5) Support extracting data in formats like BibTeX, CSV, and graph databases.
- 6) Allow the system to update trees over time as new papers are published.
- 7) Improve performance via optimization, since generating trees currently takes noticeable time.
- 8) Use LLMs to generate better abstracts instead of relying only on simple extraction methods, while managing the cost of extra LLM usage.
- 9) Improve deployment by securely handling API keys and reducing issues caused by free-tier limits, which can stop explanation generation when the quota is used up.
- 10) Support multiple users at once to prevent delays from queued tasks.

Appendix

A. Reproducibility Checklist for Local Execution

- Python dependencies installed from requirements.
- Java runtime installed for Tika.

B. Minimal Reproduction Commands for Local Execution

```
python -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt
```

```
export GROQ_API_KEY="..."  
export GEMINI_API_KEY="..."  
  
python web_app.py
```